BetterCode: Reliable

Your code is wrong, so design for that

Overview

- *Recap day 1+2+3* - readable, reusable, reproducible
  - Help with code comprehension & abstraction & execution
  - Efficient repeatable analyses
- **Reliable**: Revision
  - Help with confidence
  - Activity 1: Edge cases
  - Activity 2: Unit tests

Reliable science: peer-review process

What is reviewed carefully	What determines correctness
Writing clarity	*Code logic*
Figures	*Data processing*
Conclusions	*Implementation details*
Narrative	*Code quality*
Format / Aesthetics	*Code documentation*

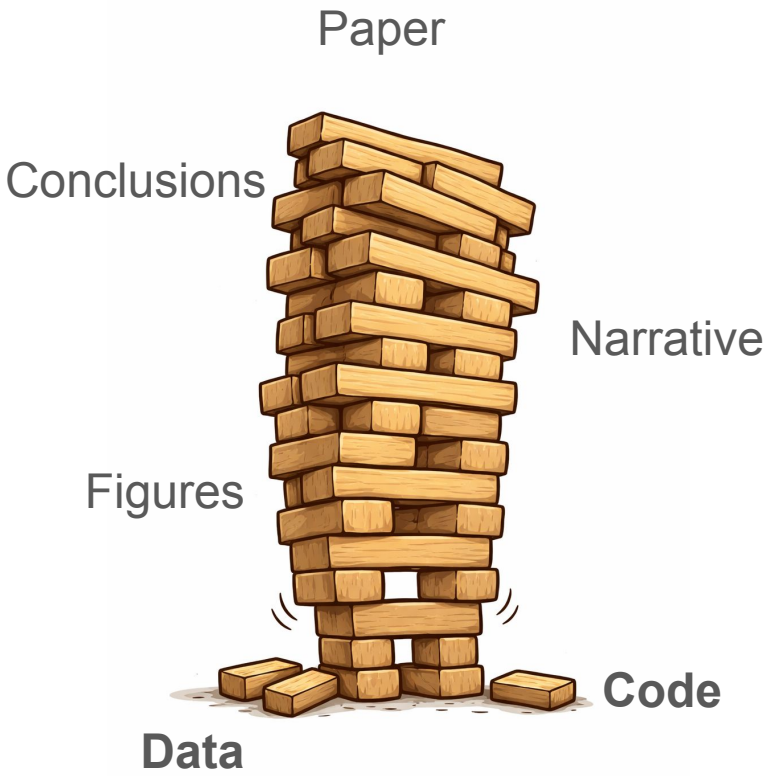
Time spent in research project:

- Writing & figures: [redacted]
- Analysis coding: [redacted]
- Code review/testing: [redacted]

Why should code be reliable?

Code errors can be “silent”

Incorrect code → incorrect science



Why should code be reliable?

VOLUME 34 · NUMBER 8 · MARCH 10, 2016

JOURNAL OF CLINICAL ONCOLOGY

ORIGINAL REPORT

This article was retracted on July 25, 2016

Inferring the Effects of Cancer Treatment: Divergent Results From Early Breast Cancer Trialists' Collaborative Group Meta-Analyses of Randomized Trials and Observational Data From SEER Registries

Katherine E. Henson, Reshma Jagsi, David Cutter, Paul McGale, Carolyn Taylor, and Sarah C. Darby

See accompanying editorial on page 772

- An error in one line of code
- The overall conclusion is still valid
- Voluntary retraction

What makes code reliable? Tried and true

- Reproducible
- Correct code produce right output for the right reasons
  - Robust analysis
- Robust code handle edge cases
  - Replicable analysis
- Reviewed and tested

The Turing Way

		Data	
		Same	Different
Analysis	Same	Reproducible	Replicable
	Different	Robust	Generalisable

Demo 1a: Silent break - Boxplot

I visualize measurements combining boxplots and jitter plot

Where is my code “wrong”?

```{r}
plot_boxplot <- function(data, x, y, fill) {
  ggplot(data, aes(x = .data[[x]], y = .data[[y]], fill = .data[[fill]])) +
    geom_boxplot(position = position_dodge(width = 0.75)) +
    geom_point(aes(group = .data[[fill]]),
               position = position_jitterdodge(jitter.width = 0.15)) +
    labs(x = x, y = y, fill = fill, color = fill) +
    theme_minimal()
}
```

```
def plot_boxplot(data, x, y, fill, figsize=(10,6)):
    figure, ax = plt.subplots(figsize=figsize)

    sns.boxplot(data=data, x=x, y=y, hue=fill,
                flierprops={'markerfacecolor': 'black'}, ax=ax)
    sns.stripplot(data=data, x=x, y=y, hue=fill,
                  dodge=True, palette='dark:black',
                  size = 6, legend=False, ax=ax)

    plt.close(figure)
    return figure
```

Demo 1b: Silent break - Moving average

I smooth a timeseries data with moving averages.

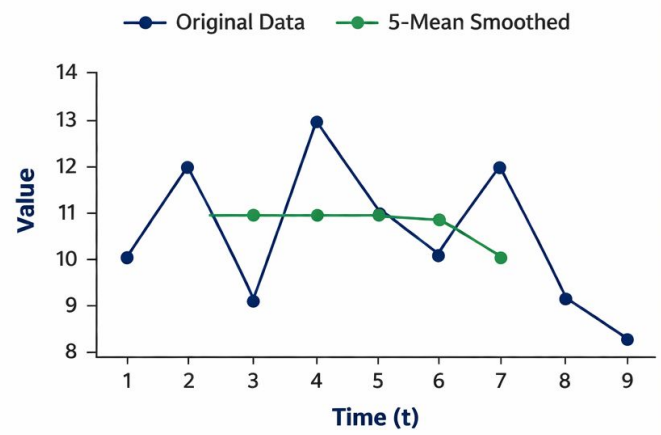
Identify what's wrong with my code

5-Mean Smoothing

Replace each point with the average of itself and the 2 neighbors on each side.

Time (t)	Original x(t)	5-Mean Smoothed y(t)
1	10	-
2	12	-
3	9	(10+12+9+13+11)/5 = **11.0**
4	13	(12+9+13+11+10)/5 = **11.0**
5	11	(9+13+11+10+12)/5 = **11.0**
6	10	(13+11+10+12+9)/5 = **11.0**
7	12	(11+10+12+9+8)/5 = **10.0**
8	9	-
9	8	-

Note: *The first 2 and last 2 points don't have enough neighbors, so they are not smoothed (shown as "-").*



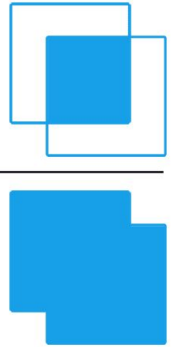
Demo 1: Silent break

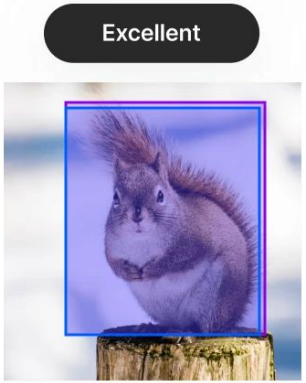
Suggestions:

- Export & inspect intermediate data, *not just* inputs → outputs
- Check distribution & sample individual data points, *not just* summary statistics
- What condition may break the code, not just this particular dataset/settings

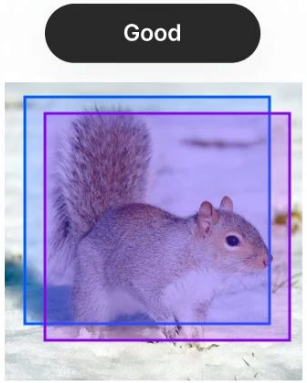
Activity 1 - Identify potential edge cases

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

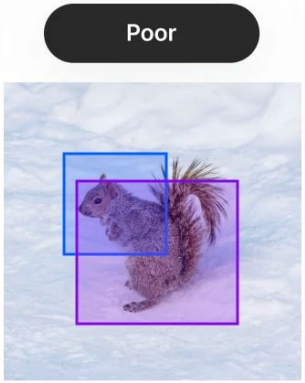




IoU= 0.95



IoU= 0.79



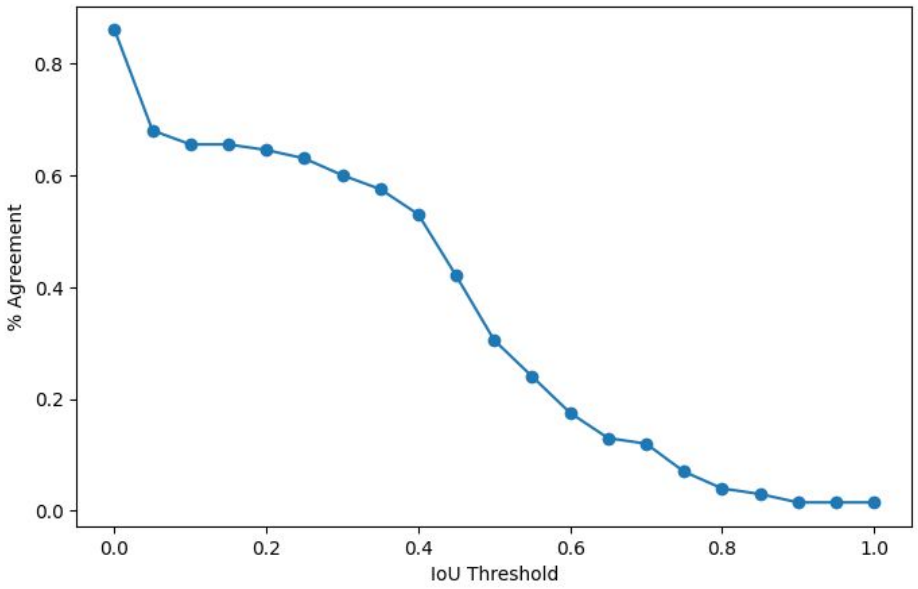
IoU= 0.45

Activity 1 - Identify potential edge cases

Two annotators segmented 200 images of cells. We compute IoU to compare the similarity of the two annotation sets.

What (potential) issues does the code have?

How can we fix it?



Activity 1 - Discussion

Which potential problems (edge cases) are easy to think of? Which are hard?

How do you check its correctness?

(Software) Code Testing

Smoke	Does it run at all?
Unit	**Does each unit function correctly?**
Integration	Do components work together?
System	Does the whole pipeline behave as expected?
Regression	Do old results still hold?

Reference: https://book.the-turing-way.org/reproducible-research/testing/testing-overview/

(Data Analysis) Code Testing

Software Code	Data Analysis Code
Pipeline-centric	Data-centric
Clear inputs → outputs	Less clear outputs
Generalizable	Implicit assumptions
Unit tests	Sanity + validation

Good habits:

- Adopt *appropriate* software code testing
- QC checkpoints
  - Same data, different figures
  - Outliers
- Peer discussion
  - Small code chunks
  - Intermediate/QC outputs

Demo 2: Write unit test (with pytest / testthat)

Write unit tests for the demo 1a.

Potential options:

- Input structure: empty, singleton, multiple
- Error handling
- Mock cases: expected output/dimensions

Activity 2 - Write your unit test

Write 2 unit tests for the code in activity 1.

- One test for a typical input
- One test for an edge case

Activity 2 - Discussion

Using AI to write (unit) tests, it is *good**

- Generate initial test scaffolding
- Fill in repetitive test cases
- Suggest edge cases
- Convert examples into tests

Weakness

- **Review the test is required**: Should be *easy* with unit tests
- Lacks intents, common sense

Take-away message

- Code becomes reliable through iteration, testing, and review from multiple angles.